

FINAL TECHNICAL REPORT

IPB Food Hub & UMKM

Platform Marketplace Digital UMKM Kampus Berbasis Web

Mata Kuliah: Keamanan Informasi (KOM1315)

Program: Ilmu Komputer, FMIPA IPB University

Semester: Genap 2025/2026

Daftar Isi

1. Ringkasan Proyek	4
1.1 Latar Belakang	4
1.2 Tujuan Final Release	4
1.3 Ruang Lingkup Implementasi	4
2. Struktur Implementasi	4
2.1 Struktur Repositori	4
2.2 Komponen Backend	5
2.3 Komponen Frontend	6
2.4 Deployment	7
3. Implementasi Keamanan	7
3.1 Authentication : Pembuktian Identitas	7
3.2 Authorization : Kontrol Akses Berbasis Peran (RBAC)	8
3.3 Accounting : Audit Logging (AAA)	8
3.4 Enkripsi Data Sensitif (AES-256-GCM)	9
3.5 Tanda Tangan Digital (RSA-2048 PSS)	9
4. Alur Utama Aplikasi	10
4.1 Registrasi dan Manajemen Akun	10
4.2 Login Multi-Peran	10
4.3 Alur Pemesanan dan Tanda Tangan Digital	10
4.4 Security Dashboard Admin	11
5. Hasil Pengujian	11
5.1 Ringkasan Hasil Unit Test	11
5.2 Cakupan Pengujian Keamanan	11
5.3 Bukti Pengujian dalam Repositori	12
5.4 Keterbatasan yang Perlu Ditingkatkan	12
6. Panduan Instalasi dan Operasional	12
6.1 Prasyarat	12
6.2 Menjalankan Backend	12
6.3 Menjalankan Frontend	13
6.4 Variabel Lingkungan Penting	13

7. Panduan Pengguna.....	13
7.1 Mahasiswa	13
7.2 Pemilik UMKM	14
7.3 Administrator	14
8. Kesimpulan	14

1. Ringkasan Proyek

1.1 Latar Belakang

IPB Food Hub adalah platform marketplace digital berbasis web yang menghubungkan mahasiswa IPB dengan pelaku UMKM kuliner kampus. Sistem dirancang untuk menyederhanakan proses pemesanan makanan sekaligus memberikan eksposur digital kepada UMKM kampus yang selama ini hanya mengandalkan penjualan langsung.

Pada fase mata kuliah Keamanan Informasi (KOM1315), fokus pengembangan diperluas ke penguatan kontrol keamanan informasi yang mencakup implementasi AAA (Authentication, Authorization, Accounting), enkripsi data sensitif dengan AES-256-GCM, tanda tangan digital RSA-2048 PSS untuk non-repudiation transaksi, serta security dashboard yang dapat dipantau administrator secara real-time.

1.2 Tujuan Final Release

Dokumen ini menyatukan hasil implementasi teknis, struktur kode sumber, kontrol keamanan, pengujian, dan panduan operasional sistem, sehingga berfungsi sebagai:

- Kompilasi artefak teknis untuk penilaian akhir mata kuliah KOM1315.
- Jembatan antara dokumen desain awal dan implementasi aktual.
- Panduan instalasi, pengoperasian, dan verifikasi fitur inti.
- Ringkasan penerapan kontrol AAA, enkripsi, dan digital signature.
- Bukti pengujian keamanan yang terdokumentasi dalam repository.

1.3 Ruang Lingkup Implementasi

Final release IPB Food Hub mencakup empat area utama yang saling terintegrasi:

- Aplikasi backend berbasis FastAPI (Python 3.12) untuk autentikasi, otorisasi berbasis peran, logika domain pemesanan, enkripsi data PII, dan audit logging.
- Aplikasi frontend berbasis React 18 dan Vite untuk antarmuka pengguna per role (mahasiswa, pemilik UMKM, admin).
- Modul keamanan terintegrasi: AES-256-GCM untuk enkripsi field-level, RSA-2048 PSS untuk tanda tangan transaksi, dan JWT HS256 untuk session management.
- Security dashboard real-time untuk administrator yang menampilkan statistik AAA, audit log, dan distribusi peran pengguna.

2. Struktur Implementasi

2.1 Struktur Repositori

Tabel 1 Struktur Repositori

Folder	Keterangan
--------	------------

01_Proposal_&_Analisis/	Proposal teknis (Proposal_Teknis.docx) dan dokumen threat modeling (Threat_Modeling.docx).
02_Design_Documents/	ERD (ERD_IPBFoodHub.png), Architecture Diagram (DOCX), Testing Plan (DOCX).
03_Source_Code/backend/	Implementasi server FastAPI: autentikasi, otorisasi, enkripsi, digital signature, audit log, semua router dan schema.
03_Source_Code/frontend/	Aplikasi React: halaman login, register, security dashboard, audit log, dan API service.
03_Source_Code/auth/	Compatibility endpoint yang me-re-export modul security, logging AAA, dan digital signature.
03_Source_Code/database/	Compatibility endpoint untuk SQLAlchemy engine, session factory, dan Base.
03_Source_Code/digital_signature/	Compatibility endpoint untuk modul tanda tangan digital RSA-2048 PSS.
04_Reports_&_Paper/	Laporan monitoring P7, laporan teknis final, dan paper ilmiah.
05_Testing/	Log unit test keamanan (unit-test-security.log) dan contoh AAA audit log (auth_activity.log).

2.2 Komponen Backend

Backend IPB Food Hub dibangun dengan FastAPI 0.111 dan Python 3.12. Struktur modul utama dipisahkan ke beberapa area:

Modul	Path	Fungsi
Authentication	app/core/security.py	JWT HS256 create/verify, bcrypt password hashing, get_current_user()
Authorization (RBAC)	app/core/security.py	require_role([...]) : FastAPI Depends yang validasi role dan catat ACCESS_DENIED

Enkripsi AES	app/core/crypto.py	AES-256-GCM encrypt()/decrypt() : field-level encryption untuk data PII
Digital Signature	app/auth/digital_signature.py	RSA-2048 sign_message() dan verify_signature() untuk non-repudiation
AAA Audit Log	app/core/logging_aaa.py	log_accounting() dan log_login_attempt() : JSON Lines ke file log
API Audit Log	app/models/audit_log.py dan app/main.py	HTTP middleware mencatat setiap request ke tabel audit_logs di PostgreSQL
Security Dashboard	app/routers/security.py	GET /api/security/stats dan /audit-logs : agregasi real-time untuk admin
Konfigurasi	app/core/config.py	Baca env vars: SECRET_KEY, ENCRYPTION_KEY, RSA key paths, DATABASE_URL

2.3 Komponen Frontend

Frontend menggunakan React 18 dan Vite dengan JavaScript (JSX). Antarmuka dipisahkan per peran pengguna untuk mempertegas jalur otorisasi.

File	Fungsi
App.jsx	Routing utama dan route guard (redirect ke /login jika tidak terautentikasi)
pages/LoginPage.jsx	Form login dengan penanganan error 401 dan 403
pages/RegisterPage.jsx	Form registrasi mahasiswa dan pemilik UMKM

pages/SecurityDashboardPage.jsx	Dashboard AAA khusus admin: statistik login, distribusi peran, top endpoint
pages/AuditLogPage.jsx	Tabel audit log real-time dengan filter method, status code, user
services/api.js	Axios instance dengan Bearer token header dan interceptor

2.4 Deployment

Komponen	Platform	Keterangan
Frontend React	Vercel Static dan CDN	https://ipb-food-hub.vercel.app
Backend FastAPI	Vercel Serverless	Serverless functions via vercel.json
Database	Neon DB (PostgreSQL)	DATABASE_URL disimpan di Vercel env var
Media Storage	Cloudinary CDN	Foto menu, profil, bukti pembayaran

3. Implementasi Keamanan

3.1 Authentication : Pembuktian Identitas

Autentikasi mendukung tiga peran pengguna: mahasiswa, pemilik_umkm, dan admin. Implementasi final mencakup komponen berikut:

Mekanisme	Algoritma	Implementasi
Password Hashing	bcrypt (work factor default)	get_password_hash() : hash satu arah; verify_password() untuk verifikasi login
Token Penerbitan	JWT HS256 (python-jose)	create_access_token() dengan payload {sub: user_id, role, exp}

Token Masa Berlaku	Configurable via env	ACCESS_TOKEN_EXPIRE_MINUTES di .env (default 1440 menit = 24 jam)
Token Validasi	jwt.decode() dan DB user check	get_current_user() validasi signature dan cek user.status == 'aktif'
Cek Status Akun	DB query per request	User dengan status='suspended' ditolak dengan 403 meski token valid

3.2 Authorization : Kontrol Akses Berbasis Peran (RBAC)

Peran	Hak Akses
mahasiswa	Browsing UMKM & menu, buat dan lacak pesanan, upload bukti pembayaran, beri rating
pemilik_umkm	CRUD UMKM milik sendiri, kelola menu & promo, konfirmasi pembayaran, balas ulasan
admin	Manajemen seluruh user & UMKM, lihat security dashboard & audit log, suspend akun

RBAC diterapkan melalui fungsi `require_role([allowed_roles])` yang dipakai sebagai FastAPI Depends di setiap endpoint. Jika role tidak sesuai, sistem menolak dengan 403 Forbidden dan mencatat event `ACCESS_DENIED` ke audit log.

3.3 Accounting : Audit Logging (AAA)

Komponen Accounting diwujudkan melalui dua lapisan logging yang saling melengkapi:

Lapisan	Mekanisme	Penyimpanan	Event
API Middleware	HTTP middleware di <code>app/main.py</code> mencatat setiap request ke <code>/api/*</code>	Tabel <code>audit_logs</code> (PostgreSQL) terindeks by <code>created_at</code> , <code>user_id</code> , <code>endpoint</code>	Semua HTTP request: <code>method</code> , <code>endpoint</code> , <code>status_code</code> , <code>IP</code> , <code>duration_ms</code>

AAA JSON Log	log_accounting() dan log_login_attempt() di core/logging_aaa.py	File logs/aaa_accounting.log dalam format JSON Lines	LOGIN_SUCCESS, LOGIN_FAILED, USER_REGISTER, ACCESS_GRANTED, ACCESS_DENIED, PROFILE_UPDATE
--------------	---	--	---

3.4 Enkripsi Data Sensitif (AES-256-GCM)

Data PII pengguna dienkripsi secara transparan menggunakan SQLAlchemy TypeDecorator bernama EncryptedString. Proses enkripsi dan dekripsi berjalan otomatis saat data ditulis ke atau dibaca dari database, tanpa perubahan pada lapisan bisnis.

Field yang Dienkripsi	Tabel	Algoritma	Detail
no_telp	users	AES-256-GCM	IV 12 byte random per nilai; format: base64(IV Ciphertext GCM_Tag)
nik	users	AES-256-GCM	Nomor Induk Kependudukan pemilik UMKM
nomor_rekening	users	AES-256-GCM	Nomor rekening bank pengguna
nomor_ewallet	users	AES-256-GCM	Nomor e-wallet pengguna

Kunci enkripsi (32 byte) disimpan sebagai ENCRYPTION_KEY di Vercel environment variable dan tidak pernah di-commit ke repositori. Integritas data dijamin oleh GCM authentication tag : ciphertext yang ditamper akan menyebabkan InvalidTag exception.

3.5 Tanda Tangan Digital (RSA-2048 PSS)

Non-repudiation transaksi diimplementasikan menggunakan RSA-2048 dengan PSS padding dan SHA-256 melalui library python-cryptography.

Aspek	Detail
-------	--------

Algoritma	RSA-2048 dan PSS padding serta SHA-256
Trigger	Setiap transaksi pesanan yang diselesaikan (status = selesai)
Payload	JSON {pesanan_id, total_harga, timestamp} dengan sort_keys=True
Penyimpanan	Base64-encoded signature di kolom tanda_tangan tabel pesanan
Verifikasi	verify_signature() di app/auth/digital_signature.py
Key Management	Private key di server (.gitignore); public key untuk verifikasi independen
Keamanan	RSA-2048 setara 112-bit symmetric; PSS lebih kuat dari PKCS#1 v1.5

4. Alur Utama Aplikasi

4.1 Registrasi dan Manajemen Akun

Pengguna baru mendaftar melalui halaman RegisterPage.jsx dengan memilih role (mahasiswa atau pemilik_umkm). Backend memvalidasi email unik, melakukan hashing password dengan bcrypt, mengenkripsi data PII dengan AES-256-GCM, lalu menerbitkan JWT dan mencatat event USER_REGISTER ke audit log.

4.2 Login Multi-Peran

Pengguna login melalui POST /api/auth/login dengan email dan password. Backend memverifikasi bcrypt hash, memeriksa status akun, menerbitkan JWT, dan mencatat LOGIN_SUCCESS atau LOGIN_FAILED ke AAA log. Token JWT disimpan di memory browser (bukan localStorage) dan disertakan sebagai Bearer header pada setiap request berikutnya.

4.3 Alur Pemesanan dan Tanda Tangan Digital

1. Mahasiswa membuat pesanan (POST /api/pesanan) : JWT divalidasi, role mahasiswa diverifikasi.
2. Mahasiswa upload bukti pembayaran (foto) ke Cloudinary.
3. Pemilik UMKM mengonfirmasi pembayaran dan mengubah status pesanan.
4. Saat status pesanan diubah menjadi 'selesai', backend menghasilkan payload JSON dan menandatangani dengan RSA-2048 PSS.

5. Signature disimpan ke kolom tanda_tangan : memberikan bukti non-repudiation transaksi.

4.4 Security Dashboard Admin

Administrator mengakses /api/security/stats untuk statistik AAA agregat (total login attempt, success rate, active users, access denied) dan /api/security/audit-logs untuk tabel audit log yang dapat difilter berdasarkan method, status_code, user_id, dan endpoint.

5. Hasil Pengujian

5.1 Ringkasan Hasil Unit Test

Suite Test	File	TC	Status
Digital Signature RSA	tests/test_digital_signature.py	6	SEMUA PASS
JWT Auth & bcrypt	tests/test_security.py	7	SEMUA PASS
AES-256-GCM Enkripsi	tests/test_crypto.py	4	SEMUA PASS
TOTAL	3 suite	17	17/17 PASS

5.2 Cakupan Pengujian Keamanan

Area Uji	Kasus yang Diuji	Status
AuthGuard JWT	Token valid, token expired, token ditamper, tanpa token, user suspended	PASS
RBAC require_role()	Role sesuai (200), role tidak sesuai (403), semua peran	PASS
RSA Digital Signature	Sign & verify valid, payload ditamper, signature palsu, empty signature	PASS
AES-256-GCM	Roundtrip encrypt/decrypt, IV unik, tamper detection (InvalidTag), None input	PASS
AAA Audit Log	LOGIN_SUCCESS, ACCESS_GRANTED, tercatat LOGIN_FAILED, ACCESS_DENIED	PASS

5.3 Bukti Pengujian dalam Repositori

Artefak	Path	Keterangan
Unit test log	05_Testing/unit-test-security.log	Output pytest 17 test, semua PASS
AAA activity log	05_Testing/auth_activity.log	Contoh JSON Lines: LOGIN_SUCCESS, LOGIN_FAILED, ACCESS_GRANTED, ACCESS_DENIED

5.4 Keterbatasan yang Perlu Ditingkatkan

- Rotasi kunci RSA secara terjadwal untuk lingkungan produksi.
- Integration test end-to-end yang mencakup seluruh alur frontend-backend.
- Rate limiting dan account lockout setelah kegagalan login berulang.
- Enkripsi tambahan untuk data gambar bukti pembayaran yang tersimpan di Cloudinary.
- Monitoring anomaly detection otomatis berdasarkan pola audit log.

6. Panduan Instalasi dan Operasional

6.1 Prasyarat

Komponen	Versi Minimum
Python	≥ 3.12
Node.js	≥ 18
PostgreSQL	≥ 14 (atau Neon DB)
pip / venv	termasuk dalam Python standar

6.2 Menjalankan Backend

6. Masuk ke folder: `cd 03_Source_Code/backend`
7. Buat virtual environment: `python -m venv venv`
8. Aktifkan: `venv\Scripts\activate` (Windows) atau `source venv/bin/activate` (Linux/Mac)
9. Install dependencies: `pip install -r requirements.txt`
10. Salin `.env.example` menjadi `.env` dan isi `DATABASE_URL`, `SECRET_KEY`, `ENCRYPTION_KEY`

11. Generate AES key: `python -c "import os,base64; print(base64.b64encode(os.urandom(32)).decode())"`
12. Generate RSA keypair: `python -m app.auth.digital_signature`
13. Jalankan server: `uvicorn app.main:app --reload --port 8000`

6.3 Menjalankan Frontend

14. Masuk ke folder: `cd 03_Source_Code/frontend`
15. Install dependencies: `npm install`
16. Buat file .env: `VITE_API_URL=http://localhost:8000`
17. Jalankan dev server: `npm run dev`

6.4 Variabel Lingkungan Penting

Variabel	Fungsi
DATABASE_URL	Connection string PostgreSQL
SECRET_KEY	Kunci rahasia untuk JWT HS256 (min 32 karakter)
ENCRYPTION_KEY	Kunci AES-256-GCM (base64 dari 32 byte random)
RSA_PRIVATE_KEY_PATH	Path ke file private.pem (default: ./keys/private.pem)
RSA_PUBLIC_KEY_PATH	Path ke file public.pem (default: ./keys/public.pem)
ACCESS_TOKEN_EXPIRE_MINUTES	Masa berlaku JWT (default: 1440 menit)
CLOUDINARY_CLOUD_NAME/_API_KEY/_API_SECRET	Konfigurasi media upload Cloudinary
ALLOWED_ORIGINS	Whitelist CORS (pisahkan dengan koma)

7. Panduan Pengguna

7.1 Mahasiswa

- Daftar akun melalui halaman Register dengan role 'mahasiswa'.
- Login dan mulai browsing direktori UMKM kampus.
- Pilih menu, masukkan ke keranjang, dan buat pesanan.
- Upload bukti pembayaran setelah melakukan transfer/QRIS.
- Pantau status pesanan di halaman Pesanan Saya.

- Berikan rating dan ulasan setelah pesanan selesai.

7.2 Pemilik UMKM

- Daftar dengan role 'pemilik_umkm' dan lengkapi profil toko.
- Kelola menu (tambah, edit, hapus) dan foto menu melalui Kelola Menu.
- Buat promo diskon melalui menu Promo.
- Pantau pesanan masuk dan konfirmasi pembayaran melalui Pesanan Masuk.
- Balas ulasan pelanggan melalui halaman ulasan.

7.3 Administrator

- Login dengan akun admin dan akses dashboard admin.
- Kelola seluruh akun pengguna (suspend, lihat detail) melalui Kelola Akun.
- Pantau statistik keamanan AAA di Security Dashboard.
- Lihat dan filter audit log API di halaman Audit Log.

8. Kesimpulan

IPB Food Hub telah dikembangkan dari rancangan awal menjadi sistem yang memiliki implementasi backend, frontend, dan kontrol keamanan yang terintegrasi penuh. Repositori akhir tidak hanya memuat source code, tetapi juga dokumentasi desain, log pengujian, dan modul keamanan yang relevan dengan capaian mata kuliah KOM1315.

Hasil ini menunjukkan bahwa kontrol AAA, bcrypt password hashing, JWT session management, enkripsi field-level AES-256-GCM, RSA-2048 PSS digital signature, dan audit logging berbasis database telah masuk ke alur utama aplikasi dan terverifikasi melalui 17 unit test yang seluruhnya lulus.